

Laporan Tugas Aktivitas Praktik Individu P1



Oleh

Desni Eria Purba

(242410101055)

KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS JEMBER

JEMBER

2025/2026

Soal 1 — Data Cuaca

1. Buat section yang menampilkan data cuaca kota Surabaya menggunakan API publik <https://wttr.in/Surabaya?format=j1> (gratis, tidak perlu API key). Tampilkan: nama kota, suhu saat ini (dalam °C), dan deskripsi cuaca. Gunakan async/await dan tampilkan loading indicator saat data sedang diambil. Bisa menggunakan api publik yang sesuai proyek masing masing

Buat section yang menampilkan data cuaca kota Jember (sesuai lokasi depot air BioAqua Lab) menggunakan API publik <https://wttr.in/Jember?format=j1> (gratis, tidak perlu API key). Tampilkan: nama kota, suhu saat ini (dalam °C), dan deskripsi cuaca. Gunakan async/await dan tampilkan loading indicator saat data sedang diambil.

```
resources > views > layouts > app.blade.php > html > head > meta
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>BioAqua Lab - @yield('title', 'Air Minum Isi Ulang')</title>
7
8   {{-- Bootstrap CSS --}}
9   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet"
10
11   {{-- Custom CSS --}}
12   <link rel="stylesheet" href="{{ asset('css/style.css') }}">
13
14   @stack('styles')
15   <meta name="csrf-token" content="{{ csrf_token() }}">
16 </head>
17 <body>
18
```

Di bagian JavaScript, dibuat sebuah fungsi async bernama ambilCuaca(). Fungsi ini menggunakan perintah fetch() untuk mengambil data dari URL:

```
fetch('https://wttr.in/Jember?format=j1')
```

Kata async/await digunakan agar program menunggu data selesai diambil sebelum melanjutkan ke langkah berikutnya. Ini penting supaya tampilan tidak error karena datanya belum ada.

```

resources > views > pages > home.blade.php > div.container.my-4 > div.card.shadow-sm > div.card-body > div#cuaca-loading
55     {{-- Section Cuaca Jember --}}
56     <div class="container my-4">
57         <div class="card shadow-sm">
58             <div class="card-body">
59                 <h5 class="card-title">🌤️ Cuaca Hari Ini – Kota Jember</h5>
60
61                 {{-- Loading indicator --}}
62                 <div id="cuaca-loading" class="text-muted">
63                     <div class="spinner-border spinner-border-sm text-primary" role="status"></div>
64                     Sedang mengambil data cuaca...
65                 </div>
66
67                 {{-- Hasil cuaca --}}
68                 <div id="cuaca-result" style="display:none;">
69                     <p class="mb-1">📍 Kota: <strong id="cuaca-kota"></strong></p>
70                     <p class="mb-1">🌡️ Suhu: <strong id="cuaca-suhu"></strong>°C</p>
71                     <p class="mb-0">☁️ Kondisi: <strong id="cuaca-deskripsi"></strong></p>
72                 </div>
73
74                 {{-- Jika error --}}
75                 <div id="cuaca-error" class="text-danger" style="display:none;">
76                     ⚠️ Gagal mengambil data cuaca.
77                 </div>
78             </div>
79         </div>
80     </div>

```

Ketika data sedang diambil (proses loading), halaman menampilkan teks "Sedang mengambil data cuaca..." beserta animasi spinner. Ini agar pengguna tahu bahwa proses sedang berjalan dan tidak mengira website-nya error.

Di kode HTML, ada dua div penting:

- `div id="cuaca-loading"` → ditampilkan saat loading
- `div id="cuaca-result"` → disembunyikan saat loading, lalu ditampilkan setelah data berhasil didapat

```

resources > views > pages > home.blade.php > ...
658
659 @push('scripts')
660 <script>
661     async function ambilCuaca() {
662         try {
663             const response = await fetch('https://wttr.in/Jember?format=j1');
664             const data = await response.json();
665
666             const suhu      = data.current_condition[0].temp_C;
667             const deskripsi  = data.current_condition[0].weatherDesc[0].value;
668             const kota       = data.nearest_area[0].areaName[0].value;
669
670             document.getElementById('cuaca-kota').textContent      = kota;
671             document.getElementById('cuaca-suhu').textContent      = suhu;
672             document.getElementById('cuaca-deskripsi').textContent = deskripsi;
673
674             document.getElementById('cuaca-loading').style.display = 'none';
675             document.getElementById('cuaca-result').style.display  = 'block';
676
677         } catch (error) {
678             document.getElementById('cuaca-loading').style.display = 'none';
679             document.getElementById('cuaca-error').style.display   = 'block';
680         }
681     }

```

Setelah data berhasil diambil, program mengambil nilai-nilai berikut dari response JSON:

- Suhu: `data.current_condition[0].temp_C`
- Deskripsi cuaca: `data.current_condition[0].weatherDesc[0].value`
- Nama kota: `data.nearest_area[0].areaName[0].value`

Lalu nilai-nilai tersebut dimasukkan ke elemen HTML menggunakan `textContent`, sehingga langsung terlihat di halaman.



Di halaman utama BioAqua Lab, muncul sebuah card bertuliskan "Cuaca Hari Ini — Kota Jember" yang menampilkan tiga informasi: Kota Jember, Suhu 22°C, dan Kondisi Cloudy. Tampilan ini muncul secara otomatis tanpa perlu me-reload halaman.

Soal 2 — Live Search Produk Air Minum

2. Buat live search yang memiliki form sederhana untuk mencari mahasiswa. Form dikirim via AJAX POST/ GET ke Laravel - halaman tidak boleh reload sama sekali. Setelah berhasil, catatan baru langsung muncul di daftar bawah tanpa refresh. Gunakan CSRF token yang benar.

Buat live search yang memiliki form sederhana untuk mencari produk air minum (barang). Form dikirim via AJAX POST/GET ke Laravel — halaman tidak boleh reload sama sekali. Setelah berhasil, hasil pencarian langsung muncul di daftar bawah tanpa refresh. Gunakan CSRF token yang benar.

```

routes > web.php > ...
9 // Public routes
10 Route::get('/', [HomeController::class, 'index']);
11 Route::view('/tentang', 'tentang');
12 Route::view('/kontak', 'pages.kontak');
13 Route::get('/hitung/{a}/{b}', fn($a, $b) => $a + $b);
14
15 // Pesanan public
16 Route::post('/pesanan', [PesananController::class, 'store']);
17
18 // Auth routes dari Breeze
19 require __DIR__.'/auth.php';
20
21 // Protected - harus login
22 Route::middleware('auth')->group(function () {
23 |   Route::get('/dashboard', [DashboardController::class, 'index']->name('dashboard'));
24 | });
25
26 // Protected - harus login + admin
27 Route::middleware(['auth', 'cek.admin']->group(function () {
28 |   Route::resource('barang', BarangController::class);
29 | });
30
31 // Live Search Barang
32 Route::get('/barang/search', [BarangController::class, 'search']->name('barang.search'));

```

Dua route ditambahkan di file web.php:

- GET /barang/search → menuju method search() di BarangController, untuk menerima dan memproses keyword pencarian
- GET /cari-produk → untuk menampilkan halaman view pencarian (barang.search)

Kedua route ini berada di dalam middleware auth, artinya hanya bisa diakses jika pengguna sudah login.

```

app > Http > Controllers > BarangController.php > ...
 9  class BarangController extends Controller
90  public function destroy(Barang $barang)
100  }
101
102  public function search(Request $request)
103  {
104      $keyword = $request->get('keyword', '');
105
106      $barangs = Barang::where('user_id', auth()->id()) // filter user login
107          ->where(function ($query) use ($keyword) {
108              $query->where('nama', 'LIKE', '%' . $keyword . '%')
109                  ->orWhere('kode', 'LIKE', '%' . $keyword . '%')
110                  ->orWhere('kategori', 'LIKE', '%' . $keyword . '%');
111          })
112          ->get();
113
114      return response()->json($barangs);
115  }
116 }

```

Di dalam controller, method search() bekerja sebagai berikut:

- Mengambil nilai keyword dari request: `$request->get('keyword', '')`
- Melakukan query ke database tabel barang dengan filter: nama, kode, atau kategori yang mengandung keyword tersebut (menggunakan LIKE)
- Hasil query dikembalikan dalam format JSON menggunakan `response()->json($barangs)`

Query juga difilter berdasarkan `user_id` dari pengguna yang sedang login, sehingga setiap pengguna hanya melihat data miliknya sendiri.

```

resources > views > barang > search.blade.php > ...
1  @extends('layouts.app')
2
3  @section('title', 'Cari Produk')
4
5  @section('content')
6  <div class="container mt-4">
7      <div class="card shadow-sm">
8          <div class="card-body">
9
10             <h5 class="card-title"> Cari Produk Air Minum</h5>
11             <p class="text-muted">Ketik nama, kode, atau kategori produk</p>
12
13             {{-- Form Live Search --}}
14             <div class="input-group mb-3">
15                 <input type="text"
16                     id="input-search"
17                     class="form-control form-control-lg"
18                     placeholder="Contoh: galon, botol, CLEO..."
19                     autocomplete="off">
20                 <span class="input-group-text"> </span>
21             </div>
22
23             {{-- Loading --}}
24             <div id="search-loading" style="display:none;" class="text-muted mb-2">
25                 <div class="spinner-border spinner-border-sm text-primary"></div>
26                 Mencari produk...
27             </div>

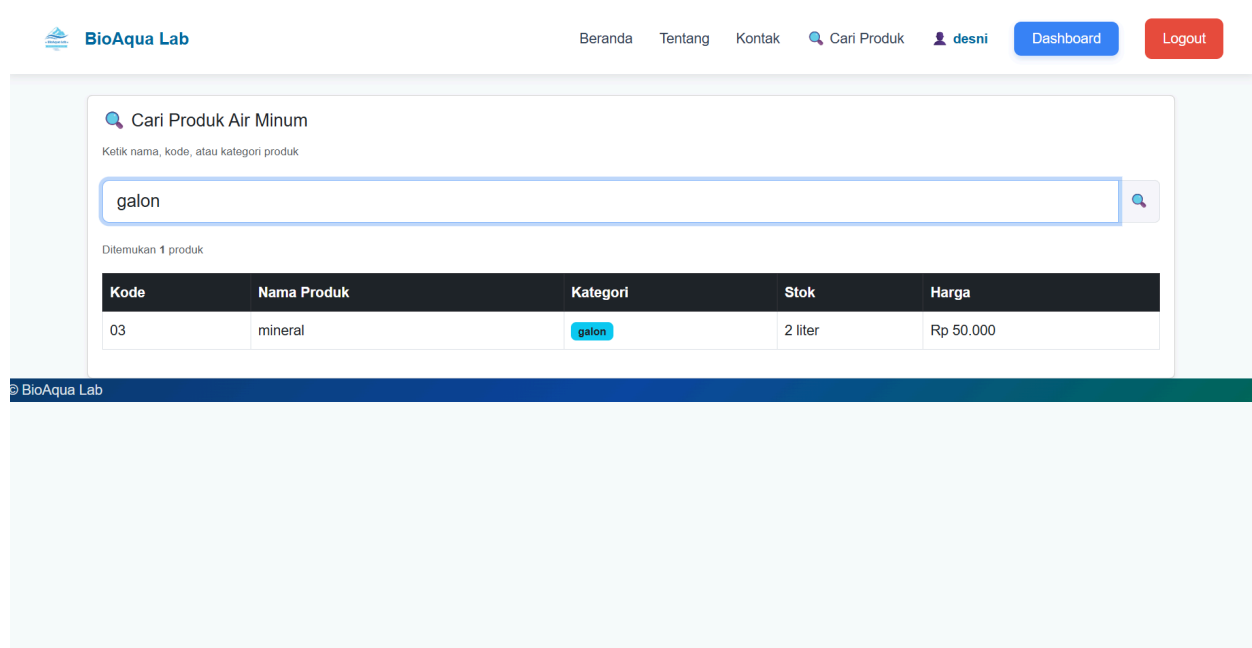
```

Di halaman pencarian, terdapat sebuah input teks dengan id="input-search". Pengguna mengetik di sini, dan secara otomatis sistem akan mengirimkan request ke endpoint /barang/search dengan keyword yang diketik. Juga terdapat elemen loading (spinner) yang muncul saat proses pencarian sedang berjalan, dan sebuah area untuk menampilkan hasil pencarian (tabel produk).

```

routes > web.php > ...
30
31 // Live Search Barang
32 Route::get('/barang/search', [BarangController::class, 'search'])->name('barang.search');
33
34 Route::get('/cari-produk', function () {
35     return view('barang.search');
36 }->name('barang.cari');

```



Di halaman Cari Produk Air Minum, pengguna mengetik kata "galon" di kolom pencarian. Hasilnya langsung muncul di bawah: tabel yang menampilkan 1 produk ditemukan, yaitu produk dengan kode 03, nama "mineral", kategori "galon", stok 2 liter, dan harga Rp 50.000. Semua ini terjadi tanpa reload halaman.

Soal 3 — Dark Mode di Navbar

3. Buat navbar untuk mengeset dark mode.

- Tambahkan dark mode ke konfigurasi Tailwind (darkMode: 'class' di tailwind.config.js)
- Buat helper function JavaScript: `setCookie`, `getCookie`, `deleteCookie`
- Implementasikan toggle dark mode: klik tombol → toggle class 'dark' di HTML → simpan ke cookie
- Terapkan cookie di awal load (dalam tag script di `<head>`) untuk mencegah flash of unstyled content
- Buat halaman pengaturan /preferensi dengan form: pilihan tema (light/dark/system), pilihan ukuran font

- f. Simpan form ke cookie via Fetch POST ke endpoint Laravel
- g. Laravel Controller: baca cookie dari \$request->cookie(), kembalikan JSON konfirmasi

Buat tombol di navbar untuk mengaktifkan dark mode dengan ketentuan:

- Tambahkan dark mode ke konfigurasi Tailwind (dilewati karena proyek pakai Bootstrap)

```
resources > views > layouts > app.blade.php > html > head
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>BioAqua Lab - @yield('title', 'Air Minum Isi Ulang')</title>
7
8   {{-- Anti flash: terapkan tema dari cookie sebelum halaman render --}}
9 <script>
10   (function() {
11     function getCookie(name) {
12       const match = document.cookie.match(new RegExp('(^| )' + name + '=[^;]+'));
13       return match ? match[2] : '';
14     }
15     if (getCookie('tema') === 'dark') {
16       document.documentElement.classList.add('dark-mode');
17     }
18   })();
19 </script>
```

Karena proyek menggunakan Bootstrap (bukan Tailwind), dark mode diterapkan menggunakan CSS class. Ketika class dark-mode ditambahkan ke elemen <html>, maka seluruh elemen akan berubah warna sesuai style yang sudah didefinisikan, antara lain:

- Background halaman menjadi #1a1a2e
 - Warna teks menjadi #e0e0e0
 - Card, navbar, form, dan alert juga menyesuaikan warna gelapnya masing-masing
- Buat helper function JavaScript: setCookie, getCookie, deleteCookie

```

public > css > style.css > ...
1520
1527 /* Dark Mode */
1528 .dark-mode {
1529     background-color: #1a1a2e !important;
1530     color: #e0e0e0 !important;
1531 }
1532 .dark-mode .card {
1533     background-color: #16213e !important;
1534     color: #e0e0e0 !important;
1535     border-color: #444 !important;
1536 }
1537 .dark-mode .navbar {
1538     background-color: #0f3460 !important;
1539 }
1540 .dark-mode .table {
1541     color: #e0e0e0 !important;
1542 }
1543 .dark-mode .form-control,
1544 .dark-mode .form-select {
1545     background-color: #16213e !important;
1546     color: #e0e0e0 !important;
1547     border-color: #444 !important;
1548 }
1549 .dark-mode .alert {
1550     background-color: #16213e !important;
1551     color: #e0e0e0 !important;
1552 }

```

Di dalam tag <head> di file app.blade.php, terdapat script kecil yang berjalan pertama kali sebelum halaman tampil. Script ini membaca cookie "tema". Jika nilainya adalah "dark", maka class dark-mode langsung ditambahkan ke elemen HTML sebelum konten apapun dirender. Ini mencegah tampilan "kedip" dari putih ke gelap saat halaman dimuat.

- Implementasikan toggle dark mode: klik tombol → toggle class 'dark-mode' di HTML → simpan ke cookie

```

resources > views > partials > navbar.blade.php > nav.navbar > div.navbar-container > ul.navbar-menu
1  <nav class="navbar">
2    <div class="navbar-container">
10     <ul class="navbar-menu">
11     </ul>
12     @endauth
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35     <li>
36         <button onclick="toggleDarkMode()" id="btn-darkmode" class="btn-darkmode">
37             🌙 Dark Mode
38         </button>
39     </li>
40 </ul>
41 </div>
42 </nav>
43
44 <style>

```

Di script navbar, terdapat tiga fungsi pembantu untuk mengelola cookie:

- `setCookie(name, value, days)` — menyimpan nilai cookie dengan waktu kadaluarsa
- `getCookie(name)` — membaca nilai cookie berdasarkan namanya
- `deleteCookie(name)` — menghapus cookie dengan mengatur kadaluarsa ke masa lalu

```

resources > views > partials > navbar.blade.php > script
139 <script>
140     function setCookie(name, value, days) {
141         const d = new Date();
142         d.setTime(d.getTime() + (days * 24 * 60 * 60 * 1000));
143         document.cookie = name + "=" + value + ";expires=" + d.toUTCString() + ";path=/";
144     }
145
146     function getCookie(name) {
147         const match = document.cookie.match(new RegExp('(^| )' + name + '=[^;]+'));
148         return match ? match[2] : '';
149     }
150
151     function deleteCookie(name) {
152         document.cookie = name + "=;expires=Thu, 01 Jan 1970 00:00:00 UTC;path=/";
153     }
154
155     function toggleDarkMode() {
156         const html = document.documentElement;
157         html.classList.toggle('dark-mode');
158
159         if (html.classList.contains('dark-mode')) {
160             setCookie('tema', 'dark', 30);
161             document.getElementById('btn-darkmode').textContent = '☀️ Light Mode';
162         } else {
163             setCookie('tema', 'light', 30);
164             document.getElementById('btn-darkmode').textContent = '🌙 Dark Mode';
165         }
166     }

```

Fungsi `toggleDarkMode()` berjalan ketika tombol di navbar diklik. Cara kerjanya:

- Toggle class `dark-mode` pada elemen `<html>`
- Jika `dark-mode` aktif → simpan cookie `tema=dark` selama 30 hari, tombol berubah teks menjadi "Light Mode"

- Jika dark-mode nonaktif → simpan cookie tema=light, tombol kembali bertuliskan "Dark Mode"
- Terapkan cookie di awal load dalam tag script di head untuk mencegah flash of unstyled content

```
PS C:\xampp\htdocs\BioAquaLab\Tugas Mandiri P1> php artisan make:controller PreferensiController

INFO Controller [C:\xampp\htdocs\BioAquaLab\Tugas Mandiri P1\app\Http\Controllers\PreferensiController.php] created successfully.

PS C:\xampp\htdocs\BioAquaLab\Tugas Mandiri P1>
```

Dibuat halaman pengaturan dengan dua pilihan:

- Tema Tampilan: Light / Dark / Ikuti System
- Ukuran Font: Kecil / Sedang / Besar

Form ini tidak di-submit secara tradisional. Melainkan menggunakan Fetch POST ke endpoint Laravel /preferensi/simpan.

- Buat halaman pengaturan /preferensi dengan form pilihan tema (light/dark/system) dan pilihan ukuran font

```

routes > web.php > ...
1  <?php
2
3  use App\Http\Controllers\DashboardController;
4  use App\Http\Controllers\BarangController;
5  use Illuminate\Support\Facades\Route;
6  use App\Http\Controllers\HomeController;
7  use App\Http\Controllers\PesananController;
8  use App\Http\Controllers\PreferensiController;
9
10 Route::get('/', [HomeController::class, 'index']);
11 Route::view('/tentang', 'tentang');
12 Route::view('/kontak', 'pages.kontak');
13 Route::get('/hitung/{a}/{b}', fn($a, $b) => $a + $b);
14
15 Route::post('/pesanan', [PesananController::class, 'store']);
16
17 require __DIR__.'/auth.php';
18
19 Route::middleware('auth')->group(function () {
20
21     Route::get('/dashboard', [DashboardController::class, 'index']->name('dashboard'));
22
23     Route::get('/barang/search', [BarangController::class, 'search']->name('barang.search'));
24     Route::get('/cari-produk', fn() => view('barang.search')->name('barang.cari'));
25
26     Route::get('/preferensi', [PreferensiController::class, 'index']->name('preferensi'));
27     Route::post('/preferensi/simpan', [PreferensiController::class, 'simpan']->name('preferensi.simpan'));
28
29 });

```

Di web.php ditambahkan dua route:

- GET /preferensi → menampilkan halaman preferensi
- POST /preferensi/simpan → memanggil method simpan() di PreferensiController

Di dalam method simpan(), Laravel membaca input tema dan font_size, kemudian membaca cookie lama (\$request->cookie('tema')), lalu mengembalikan response JSON konfirmasi berisi success: true, pesan, nilai tema baru, font_size, dan tema lama. Response juga menyertakan cookie baru yang disimpan 30 hari.

- Simpan form ke cookie via Fetch POST ke endpoint Laravel

```

resources > views > pages >  preferensi.blade.php > ...
1  @extends('layouts.app')
2
3  @section('title', 'Preferensi')
4
5  @section('content')
6  <div class="container mt-4">
7      <div class="row justify-content-center">
8          <div class="col-md-6">
9              <div class="card shadow-sm">
10                 <div class="card-header bg-primary text-white">
11                     <h5 class="mb-0"> ⚙️ Pengaturan Preferensi</h5>
12                 </div>
13                 <div class="card-body">
14
15                     {{-- Pilihan Tema --}}
16                     <div class="mb-3">
17                         <label class="form-label fw-bold">Tema Tampilan</label>
18                         <select id="pilih-tema" class="form-select">
19                             <option value="light"> ☀️ Light</option>
20                             <option value="dark"> 🌙 Dark</option>
21                             <option value="system"> 🖥️ Ikuti System</option>
22                         </select>
23                     </div>
24
25                     {{-- Pilihan Ukuran Font --}}
26                     <div class="mb-3">
27                         <label class="form-label fw-bold">Ukuran Font</label>
28                         <select id="pilih-font" class="form-select">
29                             <option value="small">Kecil</option>

```

Di web.php ditambahkan dua route:

- GET /preferensi → menampilkan halaman preferensi
- POST /preferensi/simpan → memanggil method simpan() di PreferensiController

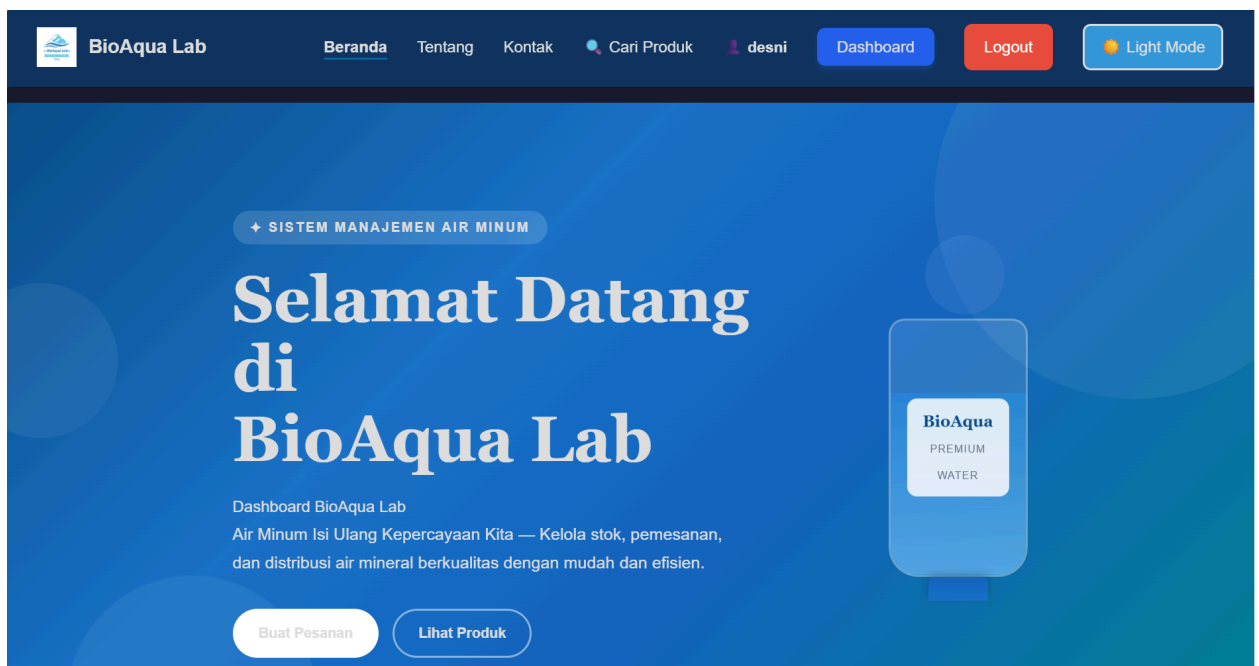
Di dalam method simpan(), Laravel membaca input tema dan font_size, kemudian membaca cookie lama (\$request->cookie('tema')), lalu mengembalikan response JSON konfirmasi berisi success: true, pesan, nilai tema baru, font_size, dan tema lama. Response juga menyertakan cookie baru yang disimpan 30 hari.

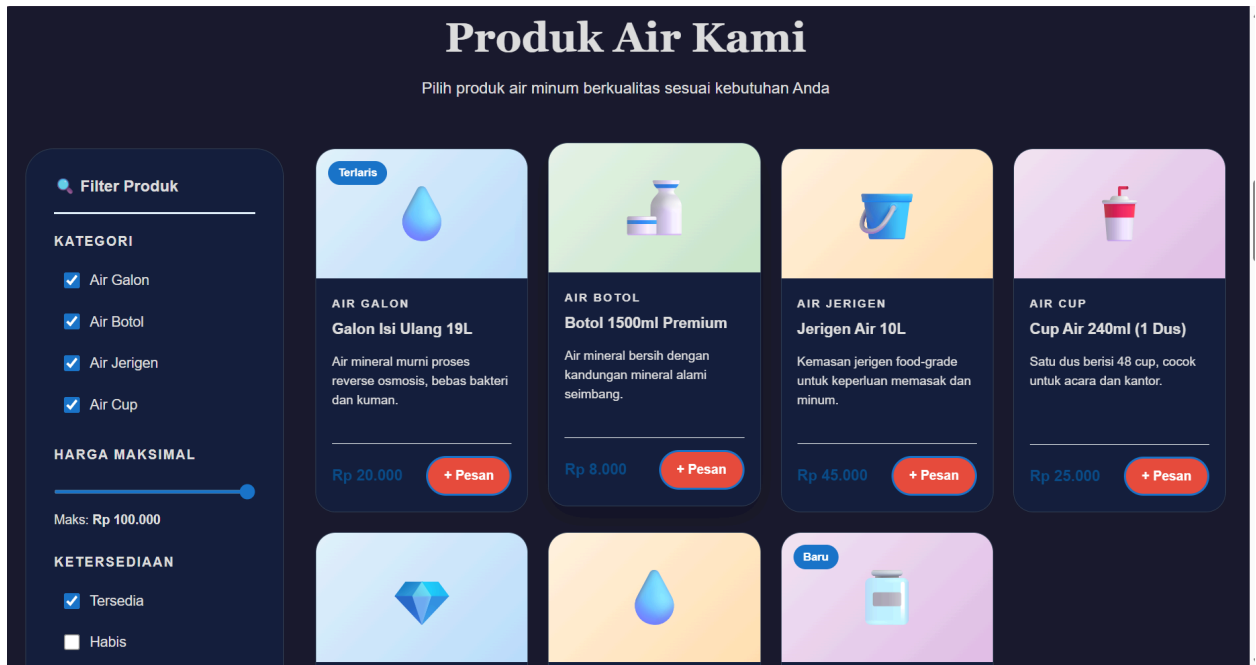
- Laravel Controller membaca cookie dari \$request->cookie() dan mengembalikan JSON konfirmasi

```

app > Http > Controllers >  PreferensiController.php >  PreferensiController >  simpan
7  class PreferensiController extends Controller
13
14  public function simpan(Request $request)
15  {
16      $tema      = $request->input('tema', 'light');
17      $fontSize  = $request->input('font_size', 'medium');
18
19      // Baca cookie lama sebagai contoh
20      $temaLama = $request->cookie('tema');
21
22      return response()->json([
23          'success'    => true,
24          'message'    => 'Preferensi berhasil disimpan!',
25          'tema'       => $tema,
26          'font_size'  => $fontSize,
27          'tema_lama'  => $temaLama,
28      ]->cookie('tema', $tema, 60 * 24 * 30)
29      ->cookie('font_size', $fontSize, 60 * 24 * 30);
30  }
31  }

```





Navbar menampilkan tombol berlabel "Dark Mode". Setelah diklik, seluruh halaman berubah ke tema gelap dan tombol berubah menjadi "Light Mode". Pilihan tema tersimpan di cookie sehingga saat halaman di-refresh, tema gelap tetap aktif.

Soal 4 — Penghitung Kunjungan Halaman

Buat section yang menghitung berapa kali pengguna mengunjungi halaman tersebut menggunakan session Laravel. Tampilkan: jumlah kunjungan, waktu kunjungan pertama, dan waktu kunjungan terakhir. Sediakan tombol "Reset Hitungan" yang menghapus data dari session dan mengulang dari awal. Bisa menggunakan case cart, sesuaikan dengan proyek

Buat section yang menghitung berapa kali pengguna mengunjungi halaman menggunakan session Laravel. Tampilkan: jumlah kunjungan, waktu kunjungan pertama, dan waktu kunjungan terakhir. Sediakan tombol "Reset Hitungan" yang menghapus data dari session dan mengulang dari awal.

```

app > Http > Controllers > HomeController.php > HomeController > index
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Barang;
6
7  class HomeController extends Controller
8  {
9      public function index()
10     {
11         $barang = Barang::all();
12
13         $kunjungan = session()->get('kunjungan', 0) + 1;
14
15         if (!session()->has('pertama_kunjungan')) {
16             session()->put('pertama_kunjungan', now()->format('d/m/Y H:i:s'));
17         }
18
19         session()->put('terakhir_kunjungan', now()->format('d/m/Y H:i:s'));
20         session()->put('kunjungan', $kunjungan);
21
22         return view('pages.home', compact('barang', 'kunjungan'));
23     }
24 }

```

Setiap kali halaman utama diakses, controller melakukan hal-hal berikut:

- Mengambil nilai kunjungan saat ini dari session: `session()->get('kunjungan', 0) + 1`
- Jika session `pertama_kunjungan` belum ada, maka diisi dengan waktu sekarang (`now()->format('d/m/Y H:i:s')`)
- Session `terakhir_kunjungan` selalu diperbarui setiap kali halaman dibuka
- Session `kunjungan` diperbarui dengan nilai yang baru

Semua data session ini kemudian dikirim ke view bersama data barang menggunakan `compact()`.

```

routes > web.php > ...
11 Route::view('/tentang', 'tentang');
12 Route::view('/kontak', 'pages.kontak');
13 Route::get('/hitung/{a}/{b}', fn($a, $b) => $a + $b);
14
15 Route::post('/pesanan', [PesananController::class, 'store']);
16
17 require __DIR__ . '/auth.php';
18
19 Route::middleware('auth')->group(function () {
20
21     Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');
22
23     Route::get('/barang/search', [BarangController::class, 'search'])->name('barang.search');
24     Route::get('/cari-produk', fn() => view('barang.search'))->name('barang.cari');
25
26     Route::get('/preferensi', [PreferensiController::class, 'index'])->name('preferensi');
27     Route::post('/preferensi/simpan', [PreferensiController::class, 'simpan'])->name('preferensi.simpan');
28
29     Route::get('/reset-kunjungan', function () {
30         session()->forget(['kunjungan', 'pertama_kunjungan', 'terakhir_kunjungan']);
31         return redirect('/')->with('success', 'Hitungan kunjungan berhasil direset!');
32     })->name('reset.kunjungan');
33
34 });
35
36 Route::middleware(['auth', 'cek.admin'])->group(function () {
37     Route::resource('barang', BarangController::class);
38 });

```

Di view, terdapat tiga card yang menampilkan:

- Total Kunjungan — angka kunjungan diambil dari session('kunjungan')
- Kunjungan Pertama — waktu diambil dari session('pertama_kunjungan')
- Kunjungan Terakhir — waktu diambil dari session('terakhir_kunjungan')

Di bawahnya ada tombol merah "Reset Hitungan" yang mengarah ke route reset.kunjungan.

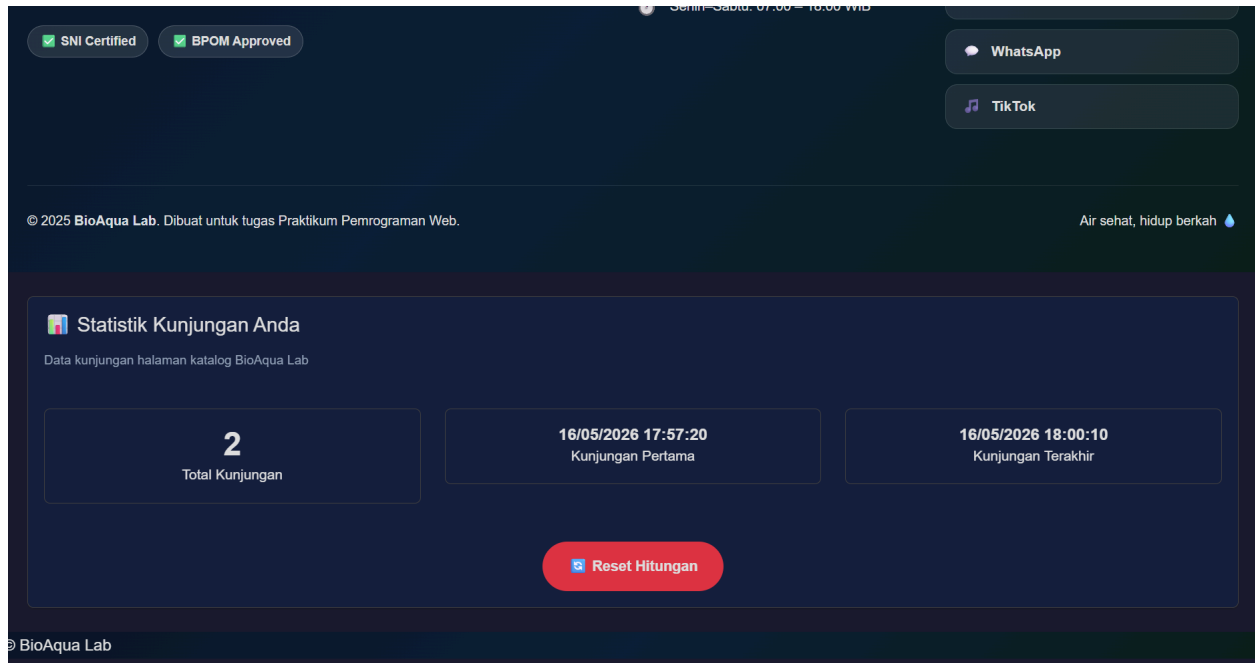
```

resources > views > pages > home.blade.php > ...
607 <div class="container my-4">
608   <div class="card shadow-sm">
609     <div class="card-body">
610       <h5 class="card-title">📊 Statistik Kunjungan Anda</h5>
611       <p class="text-muted small">Data kunjungan halaman katalog BioAqua Lab</p>
612
613       <div class="row text-center my-3">
614         <div class="col-md-4">
615           <div class="card bg-primary text-white p-3">
616             <h2 class="fw-bold mb-0">{{ session('kunjungan', 0) }}</h2>
617             <small>Total Kunjungan</small>
618           </div>
619         </div>
620         <div class="col-md-4">
621           <div class="card bg-success text-white p-3">
622             <h2 class="fw-bold mb-0">{{ session('pertama_kunjungan', '-') }}</h2>
623             <small>Kunjungan Pertama</small>
624           </div>
625         </div>
626         <div class="col-md-4">
627           <div class="card bg-info text-white p-3">
628             <h2 class="fw-bold mb-0">{{ session('terakhir_kunjungan', '-') }}</h2>
629             <small>Kunjungan Terakhir</small>
630           </div>
631         </div>
632       </div>
633
634       <div class="text-center mt-3">
635         <a href="{{ route('reset.kunjungan') }}"

```

Route GET /reset-kunjungan bekerja seperti ini:

- Menghapus ketiga session: kunjungan, pertama_kunjungan, dan terakhir_kunjungan menggunakan session()->forget([...])
- Lalu redirect ke halaman utama (/) dengan pesan sukses: "Hitungan kunjungan berhasil direset!"



Di bagian bawah halaman utama BioAqua Lab, terdapat section "Statistik Kunjungan Anda". Terlihat tiga kotak berwarna biru, hijau, dan biru muda yang menampilkan angka 2 sebagai total kunjungan, waktu kunjungan pertama (16/05/2026 17:57:20), dan waktu kunjungan terakhir (16/05/2026 18:00:10). Tombol merah "Reset Hitungan" tersedia di bawahnya.